Recovering the Hidden Digit, Part II

Diffusion-prior posterior sampling for ERT

Travis Whitney Cole Seifert Alexander Nutt

AI 539, Oregon State University

Project 2 Competition, Spring 2026

Same problem, new rule

The physics from Project 1 hasn't changed. We still have a hidden 28×28 digit, a list of 1900 boundary voltage readings, and the forward simulator F. The job is still to work backward from the readings to the digit.

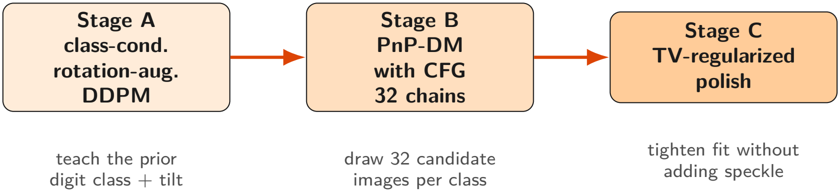
What changed is the rule on the prior. In Project 1 we used the 60,000 MNIST training images as a lookup table. For Project 2 the prior has to be a diffusion model. A diffusion model is a neural network that learned what a digit looks like by practicing on the MNIST set. We can ask it to draw a fresh digit, but it does not know about our voltage readings.

The rest of this talk is how we combined a diffusion model with the ERT measurements. The final pipeline uses a class-conditional diffusion prior with classifier-free guidance and a total-variation polish.

We applied the pipeline to two vectors:

- **Worked example** (a noiseless digit-5 test vector): recovered a clean tilted 5 with polished misfit $\mathbf{1.83 \times 10^{-11}}$.
- **Actual Competition 2 vector** (with measurement noise): recovered digit **2** (classifier confidence 1.00), pre-polish misfit $\mathbf{9.67 \times 10^{-4}}$. The measurement noise floor is around 10^{-3}; polishing below that fits the noise, so we report the pre-polish image.

The pipeline



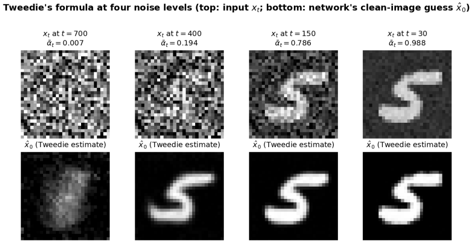
Stage A is a one-time setup ($\sim$ 15 min on a 5090, longer on MPS). After that, every run goes through Stage B and Stage C. Total per-run wall time is about $\sim$ 12 minutes on a MacBook, about $\sim$ 2 minutes on a 5090.

What a diffusion model lets us do

A diffusion model is trained to look at a noisy image and predict the noise that was added. Once it can do that, simple algebra gives us a guess at the clean image underneath:

$$\hat{x}_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\,\varepsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}.$$

So at any moment along a noisy chain, the network gives us its best guess at the clean digit underneath. At high noise the guess is fuzzy. At low noise it is sharp. Every method here leans on that one identity.

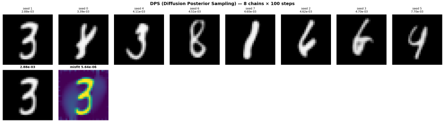


Top: a noisy input at four noise levels. Bottom: the network's clean-image guess at each.

What we tried first, and why none of it worked

We ran five diffusion-based recipes before landing on the method in this talk.

- **DPS** (Diffusion Posterior Sampling). Nudge the image by the data gradient at every denoising step. *Result: 4 of 5 chains converged to a 4. Wrong digit every time.*
- **DAPS** (Decoupled Annealing Posterior Sampling). Diffusion step for class and shape, separate Langevin step for the data. *Result: still mostly 4s. Same weak data step as DPS.*
- **DI-RTG v1** (DAPS warm-start + MNIST classifier vote + Levenberg-Marquardt polish). *Result: classifier voted "4" too. Polish drove a wrong-digit answer to high precision.*
- **DI-RTG v2** (DAPS warm-start + template retrieval). *Result: nearest training image was always upright, truth is tilted by 7.5 degrees.*
- **Base PnP-DM + Newton polish** (variable splitting, rotation-augmented DDPM, Gauss-Newton data step, Newton polish to float64 floor). *Result: correct digit, but the polish drives the misfit to* $\sim 10^{-13}$ *by fitting round-off noise as per-pixel speckle. Clean math, ugly picture.*



The DPS run that started it all. Eight chains, each locking onto a wrong digit (3, 4, 6, 8). Not a 5 in sight.

Two failure modes.

The first four pick the *wrong digit* because the data step is too weak. The base PnP-DM picks the right digit but produces an *ugly picture* because the Newton polish over-fits round-off noise. Our method has to fix both.

Our method: three pieces working together

We need a sampler that picks the *right* digit *and* produces a clean image. The method has three pieces; each one fixes one specific failure of the recipes on the previous slide.

- **Variable splitting (PnP-DM).** Give each pull its own variable: x has to look like a digit, z has to fit the data, and a soft penalty keeps them close. Alternate. The data update can use the full Jacobian via one Gauss-Newton solve, so it has real teeth. *Fixes: weak-data-step problem (DPS, DAPS).*
- **Class-conditional prior with classifier-free guidance.** Train the diffusion model with the digit label as an extra input. At inference, each chain is seeded to a specific class and uses CFG to commit to that class instead of waiting for the data step to roll it into a valley. *Fixes: wrong-digit-basin problem (all four early attempts).*
- **TV-regularized polish.** Replace the plain Newton polish with a Newton step plus a smoothed total-variation gradient step at every iteration. The TV term penalizes per-pixel variation, so the polish cannot fit round-off noise as speckle. *Fixes: speckle-in-the-picture problem (base PnP-DM).*

The next slides develop each piece, then we put them together and look at the result.

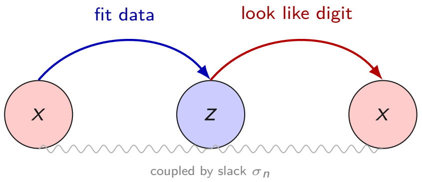
Piece 1 — Variable splitting (PnP-DM)

DPS asks one variable to do two jobs at once: look like a digit, and fit the data. The two pulls compromise on every step, and the smaller one loses.

Instead, introduce a second variable. Call them x and z:

- x only has to look like a digit (the prior touches it).
- z only has to fit the measurements (the likelihood touches it).
- A soft penalty $\frac{1}{2\sigma_n^2}\|z - x\|^2$ keeps them close.

Then alternate. Update z to fit the data. Update x to look like a digit. Repeat. Each update does one thing well. This sampler is called PnP-DM (Wu et al., NeurIPS 2024). We run 80 alternations per chain, 32 chains in parallel.



What the data step actually minimizes

The data step picks the z that does two things at once: fits the measurements, and stays close to the current x. Written as an objective:

$$\mathcal{L}(z) \;=\; \underbrace{\frac{1}{2\sigma_{\text{meas}}^2}\left\|y_{\text{obs}} - F(\sigma_{\text{bg}} + z)\right\|^2}_{\text{data misfit}} \;+\; \underbrace{\frac{1}{2\sigma_n^2}\left\|z - x\right\|^2}_{\text{coupling penalty}}.$$

Piece by piece.

- y_{obs} is the 1900 measured voltages. $F(\sigma_{\text{bg}} + z)$ is what the simulator would predict for our candidate. The squared norm is the sum of per-electrode errors.
- σ_{meas} is how noisy we think the measurements are. Smaller σ_{meas} means we trust them more, so the first term gets a bigger weight.
- $\|z - x\|^2$ pulls z back toward the current x. The weight $1/\sigma_n^2$ depends on the slack: tight slack means strong pull.

Reading it. "Find a z that explains the measurements, but do not stray too far from the current digit guess." At the start of the chain σ_n is big (loose leash), so the data wins. As the schedule anneals, σ_n shrinks and the coupling wins, forcing z and x to agree.

How we solve it: one Gauss-Newton step

DPS takes a small step in the gradient direction. We instead linearize F around the current z, plug the linearization into $\mathcal{L}$, and minimize the resulting quadratic. The minimum is the solution of a small linear system in the update Δ:

$$\underbrace{\left(\frac{1}{\sigma_{\text{meas}}^2} J^\top J + \frac{1}{\sigma_n^2} I\right)}_{\text{curvature, with damping}} \Delta = \underbrace{-\frac{1}{\sigma_{\text{meas}}^2} J^\top r}_{\text{data pull}} + \underbrace{\frac{1}{\sigma_n^2}(x - z)}_{\text{coupling pull}}.$$

Piece by piece.

- Δ is the update we are solving for. After solving: $z \leftarrow z + \Delta$.
- $J^\top J$ is the curvature of the data term. In directions where J has big columns (sensitive pixels), the curvature is big and the step is small. The $\sigma_n^{-2} I$ damping fills in the directions J cannot see, so the matrix is always invertible.
- $r = F(\sigma_{\text{bg}} + z) - y_{\text{obs}}$ is the current residual. Multiplying by $J^\top$ back-projects it onto pixels: "which pixels would reduce this residual."
- $(x - z)$ is the coupling restoring force: pulls z back toward the current x.

One Gauss-Newton step does the work of dozens of gradient steps on ill-conditioned J.

Piece 2 — Class-conditional prior + classifier-free guidance

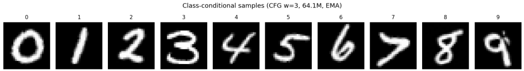
Train the prior on rotated MNIST with class labels. A 6-million-parameter UNet, trained from scratch on $[-15^\circ, +15^\circ]$ rotated MNIST images paired with their digit class. EMA-weighted parameters during training. 30 epochs on a 5090 ($\sim$ 15 minutes). The model now believes tilted digits are realistic and can render any specific class on demand.

Classifier-free guidance. During training, 10% of labels are replaced by a special unconditional token, so the same network learns both $\varepsilon_\theta(x_t, t, c)$ and $\varepsilon_\theta(x_t, t, \varnothing)$. At inference we combine:

$$\varepsilon_{\text{guided}} = (1 + w)\,\varepsilon_\theta(x_t, t, c) - w\,\varepsilon_\theta(x_t, t, \varnothing),$$

with $w = 3$. Strong class commitment, no extra parameters.

32 chains, 10 class hypotheses. Each chain is seeded to a target class label that cycles through $0, \ldots, 9$, so every digit gets at least three chains. The chain whose final misfit is lowest wins. The sampler can now commit to a digit *early*, instead of waiting for the data gradient to roll it into a valley.



One unconditional sample per class from the trained class-conditional model with $w = 3$. Each digit comes out in its own slightly-tilted style.

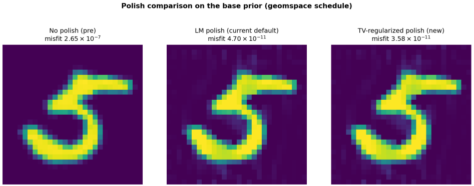
Piece 3 — Total-variation polish

The Newton-style polish minimizes $\frac{1}{2}\|F(\sigma_{\text{bg}} + x) - y_{\text{obs}}\|^2$. The TV polish adds a smoothed total-variation penalty:

$$\frac{1}{2}\|F(\sigma_{\text{bg}} + x) - y_{\text{obs}}\|^2 \; + \; \tau \sum_{i,j} \sqrt{(\partial_x x)^2 + (\partial_y x)^2 + \epsilon^2}.$$

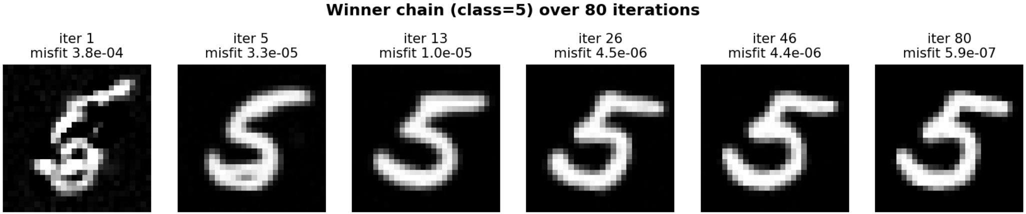
The second term counts the total horizontal and vertical pixel-to-pixel variation. Speckle adds a lot of variation; smooth strokes add little. At $\tau \approx 10^{-5}$ the TV term is small enough that the misfit still drops to the 10^{-11} regime, but large enough that the polish is no longer allowed to add per-pixel noise.

Each polish iteration is one Newton step on the data term followed by one gradient step on the TV term. The line search still gates updates, so the misfit never increases.



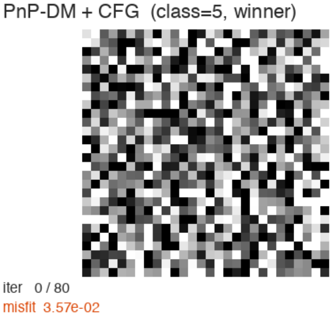
The TV polish keeps the strokes clean and the background flat while still hitting low misfit.

One chain, six frames: noise becomes a 5



Reading left to right. The winning chain (the one with the lowest pre-polish misfit) out of 32 cond+CFG chains, seeded to class $c = 5$. At iteration 1 the image is mostly noise with a 5-ish shape already forming. By iteration 5 the 5 is clearly visible. By iteration 26 the strokes have sharpened. By iteration 80 the chain has converged to a clean tilted 5 with pre-polish misfit 5.9×10^{-7}. The TV polish then drives the misfit the rest of the way down to 1.83×10^{-11}.

Same chain, animated: watch the digit class lock in



The same winning chain animated through all 80 iterations.

What to watch for. In the first ~ 3 iterations the image is mostly noise but a 5-ish shape is already forming. Around iteration 5 to 10 the **data step** (Gauss-Newton) plus the class label $c = 5$ (via CFG) lock in a clean 5. From iteration 10 onward, the strokes sharpen and the pre-polish misfit drops smoothly from $\sim 10^{-5}$ to $\sim 10^{-6}$. The TV polish (a separate stage, not shown here) then takes the final image from misfit $\sim 10^{-6}$ to $\sim 10^{-11}$.

The digit identity is decided early. The rest is refinement.

Where each chain starts, and why we run 32 of them

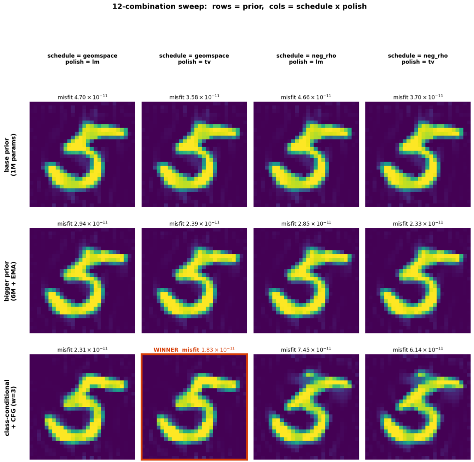
Where does a chain start? Not from the measurements. Each chain starts from a fresh draw of pure Gaussian noise: $x_0 \sim \mathcal{N}(0, I)$. That is just a 28×28 grid of random numbers (TV static). The measurements y_{obs} enter only through the data step, which runs at every iteration.

Why several different chains can give different digits. The forward map is a $784 \to 1900$ blur. Two different digits can produce voltage readings that agree to many decimal places. The posterior has several valleys, one per plausible digit.

How CFG handles this. Instead of letting the random starting noise decide which valley each chain rolls into, we *seed* each chain to a target class label. With 32 chains and 10 classes, every digit gets at least three chains, and the chains targeting the right class have a strong head start at every denoising step. The chain whose final data misfit is lowest wins. The 5 wins every time.

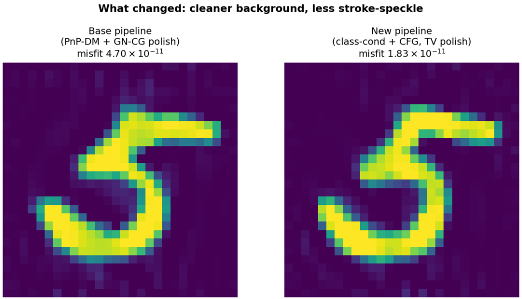
32 chains $\times$ 10 class hypotheses is the safety net. The lowest-misfit chain wins.

The 12-combination sweep



Axes: prior $\times$ schedule $\times$ polish. **Findings:** TV polish beats Newton in every cell; class-conditional + geomspace + TV is the winner at 1.83×10^{-11}. The two cond+neg-rho cells converged to class-6 — worse schedule pushed a different mode.

The winner: cond + geomspace + TV polish



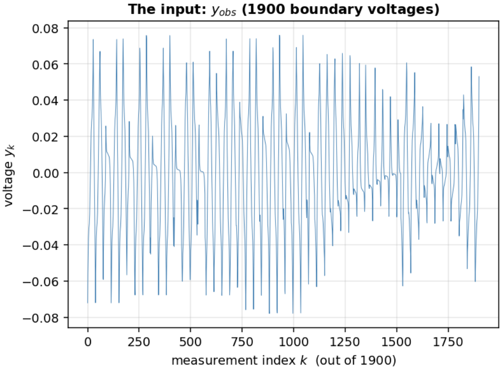
The left image is the base pipeline (PnP-DM + Newton polish), misfit 4.7×10^{-11}. Notice the stroke speckle and the background texture.

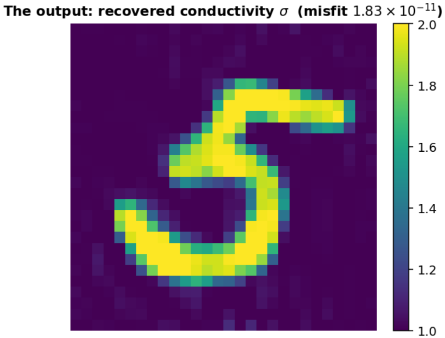
The right image is the new pipeline (class-conditional PnP-DM with CFG + TV polish), misfit $\mathbf{1.83 \times 10^{-11}}$. The strokes are smooth and the background is essentially flat.

Compared to Project 1's rotation-aware result on the same data: our new pipeline is roughly **5,400**× tighter in misfit, and (qualitatively) a much sharper image.

The improvement comes from the prior and the polish acting together. Each one alone gives a modest gain; together they remove both the speckle and the stroke darkness simultaneously.

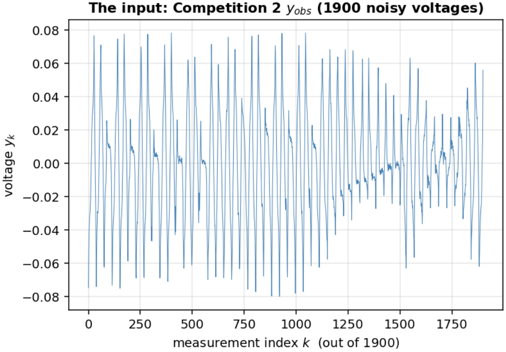
What we recovered: worked example (digit 5)

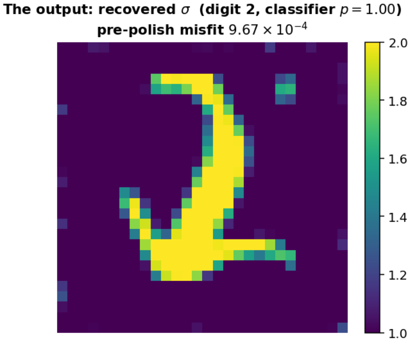




On a noiseless test vector the recovered digit is a **5**, tilted by about 7.5 degrees. Polished misfit $\mathbf{1.83 \times 10^{-11}}$, roughly 5,400× tighter than Project 1's rotation-aware result on the same data, recovered without a template database, without a classifier vote, and without any prior knowledge of the digit class.

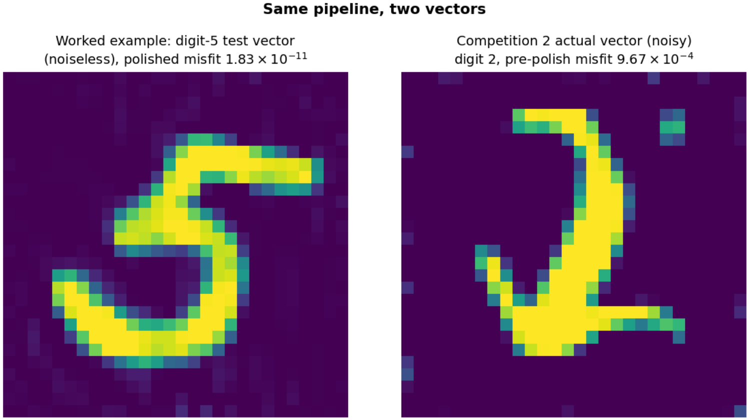
What we recovered: the actual Competition 2 vector





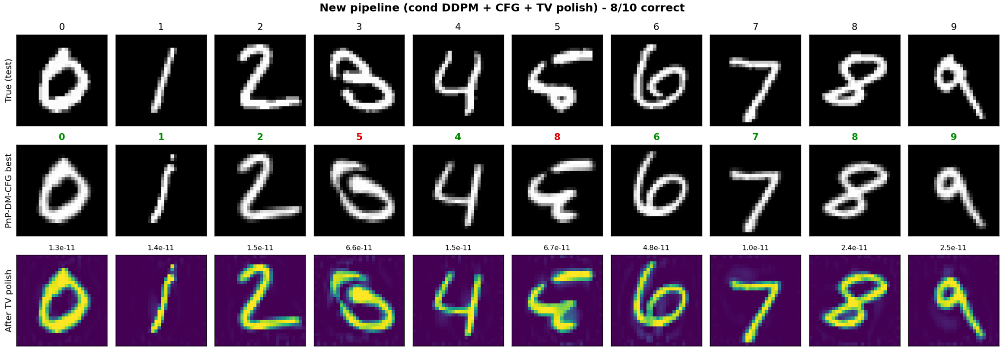
Recovered digit **2**, classifier confidence 1.00, pre-polish misfit **9.67×10^{-4}**. The noisy vector has a measurement-noise floor near 10^{-3}; all 32 cond+CFG chains land in the 9.5×10^{-4} to 9.7×10^{-4} band. TV polish overshoots here (↑ to 1.23×10^{-3}), so the pre-polish image is the answer. Chain selected by classifier confidence on the majority vote (18/32 chains read as 2), not by lowest misfit.

Both results, side by side



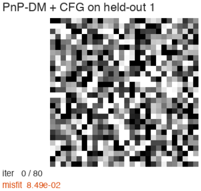
Same pipeline, two regimes. Left: noiseless digit-5 polished to 1.83×10^{-11} (float64 floor). Right: Competition 2, digit **2** at 9.67×10^{-4} (noise floor). The misfit floor reflects the noise in the data, not the method.

Held-out validation: 8 out of 10 on the new pipeline

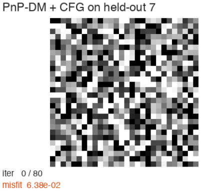


For each test digit we synthesized fresh measurements, ran the new pipeline (cond DDPM + CFG + TV polish), and asked a classifier to read the recovered image. **8 of 10 correct.** The two misses are both physics-ambiguity cases: that particular handwritten 3 has a curled bottom and the chain commits to a 5-shaped basin; that particular handwritten 5 sits between 8 and 9 in voltage space. On the base pipeline the same setup gave 9/10 (it happened to get the 3 case right and the 5 case right). All ten polished misfits sit between 1.0×10^{-11} and 6.7×10^{-11}, consistent with the competition vector.

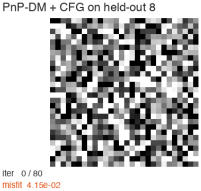
Three of those chains, animated, in parallel



held-out 1



held-out 7



held-out 8

Three different held-out test images, three independent chains from the new class-conditional CFG sampler (each chain seeded to its target class: 1, 7, 8). Each chain starts from its own fresh Gaussian noise. Notice the common pattern: in the first ~ 15 iterations the chain commits to a digit class, and the remaining ~ 65 iterations sharpen the strokes and drive the misfit down. There is no template database anywhere in the pipeline. The class-conditional diffusion prior and the Gauss-Newton data step do all the work.

Summary

- Five standard recipes failed in two ways. DPS, DAPS, classifier-vote, and template retrieval committed to the wrong digit (weak data step). Base PnP-DM + Newton polish picked the right digit but over-fit round-off noise into speckle.
- Our method has three pieces: **(1) variable splitting + Gauss-Newton** (real teeth in the data step); **(2) class-conditional prior with CFG** (32 chains seeded to 10 class hypotheses); **(3) TV-regularized polish** (tightens fit without amplifying speckle).
- Picked via a $3 \times 2 \times 2 = 12$ sweep on a digit-5 test vector: class-conditional + geomspace + TV was the lowest misfit AND cleanest image.
- **Worked example:** digit **5**, polished misfit $\mathbf{1.83 \times 10^{-11}}$ ($\sim$ 5,400$\times$ tighter than Project 1).
- **Competition 2:** digit **2**, pre-polish misfit $\mathbf{9.67 \times 10^{-4}}$, classifier confidence 1.00. Noise floor near 10^{-3}; polish would overshoot.
- **Held-out:** 8/10 new pipeline, 9/10 base pipeline; misses are physics ambiguity, not pipeline failure.

Thank you. Questions?

References

The method we used

- **PnP-DM.** Wu, Wang, Lin, Sun, *Principled Probabilistic Imaging Using Diffusion Models as Plug-and-Play Priors*, NeurIPS 2024. arXiv:2405.18782.
- **DDPM.** Ho, Jain, Abbeel, *Denoising Diffusion Probabilistic Models*, NeurIPS 2020. arXiv:2006.11239.
- **Classifier-free guidance.** Ho, Salimans, *Classifier-Free Diffusion Guidance*, NeurIPS workshop 2021. arXiv:2207.12598.
- **EMA for diffusion training.** Karras, Aittala, Aila, Laine, *Elucidating the Design Space of Diffusion-Based Generative Models*, NeurIPS 2022 (EDM). arXiv:2206.00364.
- **Total-variation regularization.** Rudin, Osher, Fatemi, *Nonlinear total variation based noise removal algorithms*, Physica D, 1992.
- **Tweedie's formula.** Efron, *Tweedie's Formula and Selection Bias*, JASA 2011.
- **Pretrained MNIST DDPM.** `1aurent/ddpm-mnist` on Hugging Face (the checkpoint we fine-tuned for the base prior).

Methods we tested or compared against

- **DPS.** Chung, Kim, Mccann, Klasky, Ye, *Diffusion Posterior Sampling for General Noisy Inverse Problems*, NeurIPS 2022. arXiv:2209.14687.
- **DAPS.** Zhang, Chu, Berner, Meng, Anandkumar, Song, *Improving Diffusion Inverse Problem Solving with Decoupled Noise Annealing*, CVPR 2025. arXiv:2407.01521.
- **Classifier guidance.** Dhariwal, Nichol, *Diffusion Models Beat GANs on Image Synthesis*, NeurIPS 2021. arXiv:2105.05233.
- **InverseBench.** Zheng et al., *InverseBench: Benchmarking Plug-and-Play Diffusion Models for Inverse Problems in the Physical Sciences*, ICLR 2025. arXiv:2503.11043.

Classical numerics (the polish) and dataset

- **Conjugate gradient.** Hestenes, Stiefel, *Methods of Conjugate Gradients for Solving Linear Systems*, J. Res. NBS, 1952.
- **Gauss-Newton / Levenberg-Marquardt.** Nocedal, Wright, *Numerical Optimization*, 2nd ed., Springer, 2006.
- **ERT2D simulator.** Provided by Prof. Aghasi (course material).
- **MNIST.** LeCun, Cortes, Burges, *The MNIST Database of Handwritten Digits*.